

PYTHON STANDARD LIBRARY

The **os** Module

A Complete Technical Reference

The `os` module provides a portable way of using operating system dependent functionality such as reading or writing to the file system, managing processes, working with environment variables, and manipulating paths. Most functions in this module wrap C library or system calls and behave consistently across Windows, Linux, and macOS wherever possible.

11 SECTIONS

56 FUNCTIONS COVERED

devforge

01 Module Basics & Introspection

Before doing anything else, `os` exposes constants that describe the current platform. These are useful for writing cross-platform code.

`os.name`

A string identifying the operating system: 'posix' for Linux/macOS, 'nt' for Windows.

```
import os
print(os.name)
# 'posix' on Linux/macOS, 'nt' on Windows
```

`os.sep`

The character used by the OS to separate path components ('/' on POSIX, '\\' on Windows).

```
print(os.sep)
# '/' on Linux/macOS
```

`os.linesep`

The line terminator used on the current platform ('\n' on POSIX, '\r\n' on Windows).

```
print(repr(os.linesep))
# '\n' on Linux/macOS
```

`os.environ`

A mapping object representing the process environment variables. Changes made to it are reflected in subprocesses.

```
print(os.environ['HOME'])
os.environ['MY_VAR'] = '1'
```

01 Module Basics & Introspection (cont.)

`os.getenv(key, default=None)`

Returns the value of an environment variable, or a default if not set. Safer than indexing `os.environ` directly.

```
path = os.getenv('PATH')
debug = os.getenv('DEBUG', 'false')
```

02 Working Directory

Every process has a current working directory. These functions let you inspect and change it.

`os.getcwd()`

Returns a string representing the current working directory.

```
cwd = os.getcwd()
print(cwd)
# /home/user/project
```

`os.chdir(path)`

Changes the current working directory to the given path. Raises `FileNotFoundError` if the path does not exist.

```
os.chdir('/tmp')
print(os.getcwd())
# /tmp
```

03 Directory Listing & Traversal

os provides several ways to list directory contents, from simple name lists to rich file metadata via scandir, plus recursive traversal with walk.

os.listdir(path='.')

Returns a list of entry names (files and directories) in the given path, excluding '.' and '..'.

```
entries = os.listdir('.')
print(entries)
# ['main.py', 'data', 'README.md']
```

os.scandir(path='.')

Returns an iterator of DirEntry objects, which cache file type and stat info. Much faster than listdir + stat in loops.

```
with os.scandir('.') as it:
    for entry in it:
        if entry.is_file():
            print(entry.name, entry.stat().st_size)
```

os.walk(top, topdown=True)

Generates the file names in a directory tree by walking either top-down or bottom-up. Yields a 3-tuple (dirpath, dirnames, filenames) for each directory.

```
for dirpath, dirnames, filenames in os.walk('.'):
    for f in filenames:
        print(os.path.join(dirpath, f))
```

os.fwalk(top='.')

Like os.walk but also yields a file descriptor for each directory, useful for avoiding race conditions in security-sensitive code.

```
for dirpath, dirnames, filenames, dirfd in os.fwalk('.'):
    print(dirpath, dirfd)
```

04 Creating & Removing Directories

Functions for creating, renaming, and deleting directories, including recursive variants for nested paths.

```
os.mkdir(path, mode=0o777)
```

Creates a single directory. Raises `FileExistsError` if it already exists, and `FileNotFoundError` if a parent is missing.

```
os.mkdir('new_folder')
```

```
os.makedirs(name, mode=0o777, exist_ok=False)
```

Creates a directory recursively, making all necessary parent directories. Set `exist_ok=True` to avoid errors if it already exists.

```
os.makedirs('project/data/raw', exist_ok=True)
```

```
os.rmdir(path)
```

Removes a single empty directory. Raises `OSError` if the directory is not empty.

```
os.rmdir('empty_folder')
```

```
os.removedirs(name)
```

Removes a directory and then recursively removes each parent directory, as long as they are empty.

```
os.removedirs('project/data/raw')
```

05 File Operations

Low-level operations for renaming, removing, and linking files. For higher-level copy operations, see the `shutil` module.

`os.rename(src, dst)`

Renames the file or directory `src` to `dst`. On some platforms, this fails if `dst` already exists.

```
os.rename('draft.txt', 'final.txt')
```

`os.replace(src, dst)`

Like `rename`, but will silently overwrite `dst` if it already exists, on all platforms. Prefer this over `rename` for safety.

```
os.replace('config.new', 'config.json')
```

`os.remove(path)`

Removes (deletes) a file. Raises `IsADirectoryError` if `path` is a directory; use `rmdir` for that instead.

```
os.remove('temp.log')
```

`os.unlink(path)`

An alias for `os.remove()`, following POSIX naming conventions. Behaves identically.

```
os.unlink('old_data.bin')
```

05 File Operations (cont.)

`os.link(src, dst)`

Creates a hard link pointing to `src` named `dst`. Both names then refer to the same inode.

```
os.link('original.txt', 'hardlink.txt')
```

`os.symlink(src, dst)`

Creates a symbolic (soft) link named `dst` pointing to `src`.

```
os.symlink('/data/config.json', 'config_link.json')
```

`os.readlink(path)`

Returns the path that a symbolic link points to.

```
target = os.readlink('config_link.json')  
print(target)
```

`os.truncate(path, length)`

Truncates the file to at most `length` bytes, without needing to open it first.

```
os.truncate('bigfile.dat', 1024)
```


06 File Status & Metadata

`os.stat` and related functions expose detailed file metadata such as size, permissions, and timestamps.

`os.stat(path)`

Returns a `stat_result` object with fields like `st_size`, `st_mtime`, `st_mode`, and `st_ino` describing the file.

```
info = os.stat('file.txt')
print(info.st_size)
print(info.st_mtime)
```

`os.chmod(path, mode)`

Changes the permission bits of a file. Mode is typically expressed in octal, e.g. `0o644`.

```
os.chmod('script.sh', 0o755)
```

`os.chown(path, uid, gid)`

Changes the owner and group id of a path. Available only on POSIX systems.

```
os.chown('shared.txt', 1000, 1000)
```

`os.utime(path, times=None)`

Sets the access and modified times of a file. If `times` is `None`, sets both to the current time.

```
import time
now = time.time()
os.utime('file.txt', (now, now))
```

06 File Status & Metadata (cont.)

`os.access(path, mode)`

Checks whether the current process has the requested access (`os.R_OK`, `os.W_OK`, `os.X_OK`, `os.F_OK`) to a path.

```
if os.access('script.sh', os.X_OK):  
    print('Executable')
```

07 Path Manipulation (os.path)

The `os.path` submodule handles path string manipulation without touching the file system — joining, splitting, and normalizing paths portably.

`os.path.join(path, *paths)`

Joins path components intelligently using the correct separator for the current OS.

```
p = os.path.join('data', 'raw', 'file.csv')
print(p)
# 'data/raw/file.csv'
```

`os.path.split(path)`

Splits a path into a (head, tail) pair, where tail is the last component.

```
head, tail = os.path.split('/data/raw/file.csv')
print(head, tail)
# /data/raw file.csv
```

`os.path.splitext(path)`

Splits a path into (root, extension), useful for changing file extensions.

```
root, ext = os.path.splitext('archive.tar.gz')
print(root, ext)
# archive.tar .gz
```

`os.path.basename(path)` / `dirname(path)`

`basename` returns the final component of the path; `dirname` returns everything before it.

```
os.path.basename('/data/raw/file.csv') # 'file.csv'
os.path.dirname('/data/raw/file.csv')  # '/data/raw'
```

07 Path Manipulation (os.path) (cont.)

os.path.abspath(path)

Returns the absolute, normalized version of a path, resolving it relative to the current working directory.

```
print(os.path.abspath('data/file.csv'))  
# /home/user/project/data/file.csv
```

os.path.exists(path)

Returns True if the path refers to an existing file or directory, following symlinks.

```
if os.path.exists('config.json'):  
    print('Found it')
```

os.path.isfile(path) / isdir(path)

Check whether a path is a regular file or a directory, respectively.

```
os.path.isfile('file.txt') # True  
os.path.isdir('folder')   # True
```

os.path.getsize(path)

Returns the size of a file in bytes.

```
size = os.path.getsize('video.mp4')  
print(f'{size / 1e6:.2f} MB')
```

07 Path Manipulation (os.path) (cont.)

`os.path.normpath(path)`

Normalizes a path by collapsing redundant separators and up-level references like '..'.

```
print(os.path.normpath('data/../data../file.csv'))  
# data/file.csv
```

08 Environment Variables

Reading and modifying the environment that the current process (and any subprocesses it spawns) sees.

`os.environ` (read)

A dict-like object of all environment variables visible to the process.

```
for key, value in os.environ.items():  
    print(key, '=', value)
```

`os.environ` (write)

Assigning to `os.environ` sets an environment variable for the current process and any children it spawns.

```
os.environ['API_KEY'] = 'secret-token'
```

`os.putenv(key, value)`

Sets an environment variable at the OS level directly. Prefer modifying `os.environ` instead, since it keeps Python's cache in sync.

```
os.putenv('LANG', 'en_US.UTF-8')
```

`os.unsetenv(key)`

Removes an environment variable at the OS level. Prefer `del os.environ[key]` for consistency.

```
del os.environ['TEMP_FLAG']
```

09 Process Management

os provides low-level primitives for spawning, controlling, and communicating with child processes. For most use cases, the higher-level subprocess module is recommended instead.

os.getpid() / os.getppid()

Returns the process ID of the current process, or its parent process.

```
print('PID:', os.getpid())
print('Parent PID:', os.getppid())
```

os.system(command)

Executes a shell command in a subshell and returns its exit status. Simple but unsafe with untrusted input.

```
exit_code = os.system('ls -la')
```

os.kill(pid, sig)

Sends a signal to a process, identified by its PID. Commonly used with signal.SIGTERM or SIGKILL.

```
import signal
os.kill(1234, signal.SIGTERM)
```

os.fork()

Creates a child process by duplicating the current process. Returns 0 in the child and the child's PID in the parent. POSIX only.

```
pid = os.fork()
if pid == 0:
    print('Child process')
else:
    print('Parent, child PID:', pid)
```

09 Process Management (cont.)

`os.wait()`

Waits for a child process to finish and returns a tuple of (pid, exit_status).

```
pid, status = os.wait()
```

`os.cpu_count()`

Returns the number of logical CPUs in the system, or None if undetermined. Useful for sizing thread/process pools.

```
print('CPUs available:', os.cpu_count())
```


10 File Descriptors & Low-Level I/O

Below Python's high-level file objects, `os` exposes raw file descriptor operations mirroring POSIX system calls.

```
os.open(path, flags, mode=0o777)
```

Opens a file and returns a raw file descriptor (an integer), rather than a Python file object.

```
fd = os.open('data.bin', os.O_RDONLY)
```

```
os.read(fd, n)
```

Reads at most `n` bytes from the file descriptor `fd`, returning bytes.

```
data = os.read(fd, 1024)
```

```
os.write(fd, data)
```

Writes the bytes in `data` to file descriptor `fd`, returning the number of bytes written.

```
written = os.write(fd, b'hello')
```

```
os.close(fd)
```

Closes the file descriptor `fd`. Always pair this with `os.open` to avoid descriptor leaks.

```
os.close(fd)
```

10 File Descriptors & Low-Level I/O (cont.)

`os.pipe()`

Creates a pipe and returns a pair of file descriptors (`read_fd`, `write_fd`) for inter-process communication.

```
r, w = os.pipe()
```

11 Useful Constants

Flags and mode constants frequently used with `os.open`, `os.access`, and related functions.

`os.O_RDONLY` / `O_WRONLY` / `O_RDWR`

Flags controlling how a file is opened: read-only, write-only, or read-write.

```
fd = os.open('f.txt', os.O_RDWR | os.O_CREAT)
```

`os.O_CREAT` / `O_APPEND` / `O_TRUNC`

Additional open flags: create the file if missing, append on write, or truncate to zero length.

```
flags = os.O_WRONLY | os.O_CREAT | os.O_APPEND
```

`os.F_OK` / `R_OK` / `W_OK` / `X_OK`

Mode values passed to `os.access()` to check existence, read, write, or execute permission.

```
os.access('file.txt', os.R_OK | os.W_OK)
```

`os.curdir` / `os.pardir`

Portable string constants for the current directory ('.') and parent directory ('..').

```
print(os.curdir, os.pardir)  
# . ..
```

Best Practices & Recommendations

Practical guidance for using the os module safely and idiomatically.

- 01 Prefer `os.path.join()` over manual string concatenation with `'/'` to keep code cross-platform.
- 02 Use `os.makedirs(path, exist_ok=True)` instead of checking existence manually before creating directories.
- 03 Prefer the `subprocess` module over `os.system()` when running shell commands with untrusted input.
- 04 Use `os.scandir()` instead of `os.listdir()` when you also need file type or stat information — it avoids redundant system calls.
- 05 Always close file descriptors opened with `os.open()` using `os.close()`, or use them inside a `try/finally` block.
- 06 On modern codebases, consider `pathlib.Path` alongside `os` for more expressive, object-oriented path handling.